PROGRAMMING IN JAVA

ASSIGNMENT 2 – CO2

EXPERIMENTAL ANALYSIS AND IMPLEMENTATION REPORT

Submitted by

Madaraboina Gayathri

Register Number: 192472074

Department of Computer Science and Engineering (CSE – AI)

SIMATS Engineering

Java Version: 26.0.1

Academic Year: 2026–2027

Declaration and Submission Details

This report documents the implementation, execution and analysis of the seven questions in Programming in Java – Assignment 2 (CO2). Experimental values are transcribed from the terminal outputs supplied during the work. Timing and heap measurements are machine- and JVM-dependent and should be interpreted as observations from the recorded runs.

GitHub Repository: To be added by the student before submission.

Note: The assignment requires a working GitHub repository link, Java source files, input files where applicable, output screenshots, raw experimental data and at least one progressive commit per question.

Table of Contents

• 1. Introduction and Objectives

• 2. Q1 – String, StringBuilder and StringBuffer

• 3. Q2 – ArrayList, LinkedList and ArrayDeque

• 4. Q3 – HashSet and the hashCode–equals Contract

• 5. Q4 – HashMap, TreeMap and LinkedHashMap

• 6. Q5 – Generics and Comparator-based Leaderboard

• 7. Q6 – PriorityQueue and Hospital Triage

• 8. Q7 – Diagnose, Redesign and Validate

• 9. Overall Conclusion

• 10. References

1. Introduction and Objectives

The purpose of this assignment is to analyse Java strings and the Collections Framework through practical scenarios. The work follows the engineering cycle: predict, design, implement, measure, analyse, evaluate, diagnose, redesign and validate.

The seven scenarios cover text construction, playlist data structures, object equality and hashing, map selection for inventory, generic ranking, priority-based patient triage and an evidence-based redesign.

Objectives

• Compare String, StringBuilder and StringBuffer under different text-generation workloads.

• Evaluate list and deque implementations for playlist operations and different collection sizes.

• Demonstrate the equals() and hashCode() contract using HashSet.

• Compare map implementations for lookup speed and report ordering.

• Build a reusable generic leaderboard using Comparator.

• Implement severity-based triage with FIFO fairness among equal priorities.

• Diagnose a measured limitation and validate a redesigned data-structure choice.

2. Q1 – Building Text Under Load: String, StringBuilder or StringBuffer (6 Marks)

Scenario: A railway reservation system constructs e-tickets by combining booking fields. The experiment compares immutable String concatenation with mutable StringBuilder and StringBuffer.

2.1 Workloads

Workload	Tickets	Fields per ticket
Small workload	1	6
Many tickets	10,000	50
Many fields	1,000	200

2.2 Execution-time observations

Workload	String (ms)	StringBuilder (ms)	StringBuffer (ms)
Small – run 1	0.027	0.019	0.025
Many tickets – run 1	34.427	10.403	11.512
Many fields – run 1	54.348	3.531	3.316
Small – run 2	0.010	0.013	0.012
Many tickets – run 2	38.179	12.421	10.908
Many fields – run 2	47.931	2.772	3.784

2.3 Approximate heap-change observations

Workload / run	String (bytes)	StringBuilder (bytes)	StringBuffer (bytes)
Small – run 1	83,904	62,928	62,928
Many tickets – run 1	60,525,224	21,479,736	20,197,912
Many fields – run 1	45,090,832	4,454,192	4,518,680
Small – run 2	83,904	62,928	62,928
Many tickets – run 2	45,615,120	21,480,224	21,733,760
Many fields – run 2	41,945,104	4,454,192	4,518,680

The heap figures are approximate changes in used heap around an execution. They are not exact allocation counts and can be affected by garbage collection, JIT compilation and JVM heap management. The two timing runs used different repetition counts (five and ten), so they are useful observations but are not strictly equivalent repeated-trial averages.

2.4 Object creation and complexity

String is immutable. Repeated concatenation in a loop creates intermediate String results. In the 1,000-ticket × 200-field workload there are 200,000 field-loop iterations; this provides a lower-bound illustration of repeated concatenation results, not an exact JVM object count. Compiler optimisations and intermediate expressions can affect the actual number.

Approach	Expected construction behaviour	Complexity discussion
String with repeated +	New String results may be created repeatedly	Can approach $O(n^2)$ character copying as a growing string is repeatedly copied
StringBuilder	Appends to a mutable buffer	Approximately O(n) total append work, with occasional resizing
StringBuffer	Mutable buffer with synchronized methods	Approximately O(n) total append work plus synchronization overhead

2.5 Prediction, observation and design decision

The larger workloads showed substantially higher execution time and approximate heap change for repeated String concatenation. StringBuilder and StringBuffer were faster in the larger workloads. The small workload showed very small differences.

Recommendation: use StringBuilder for constructing an individual ticket in a single thread. For a shared mutable booking log, StringBuffer provides synchronized operations, although a dedicated thread-safe logging mechanism is usually a stronger production design. String is suitable for fixed or small text.

2.6 Alternative concept

StringJoiner or String.join() can build text from a collection of fields and separators. String.format() can improve formatting readability but may add formatting overhead. The choice should be measured for the actual ticket format.

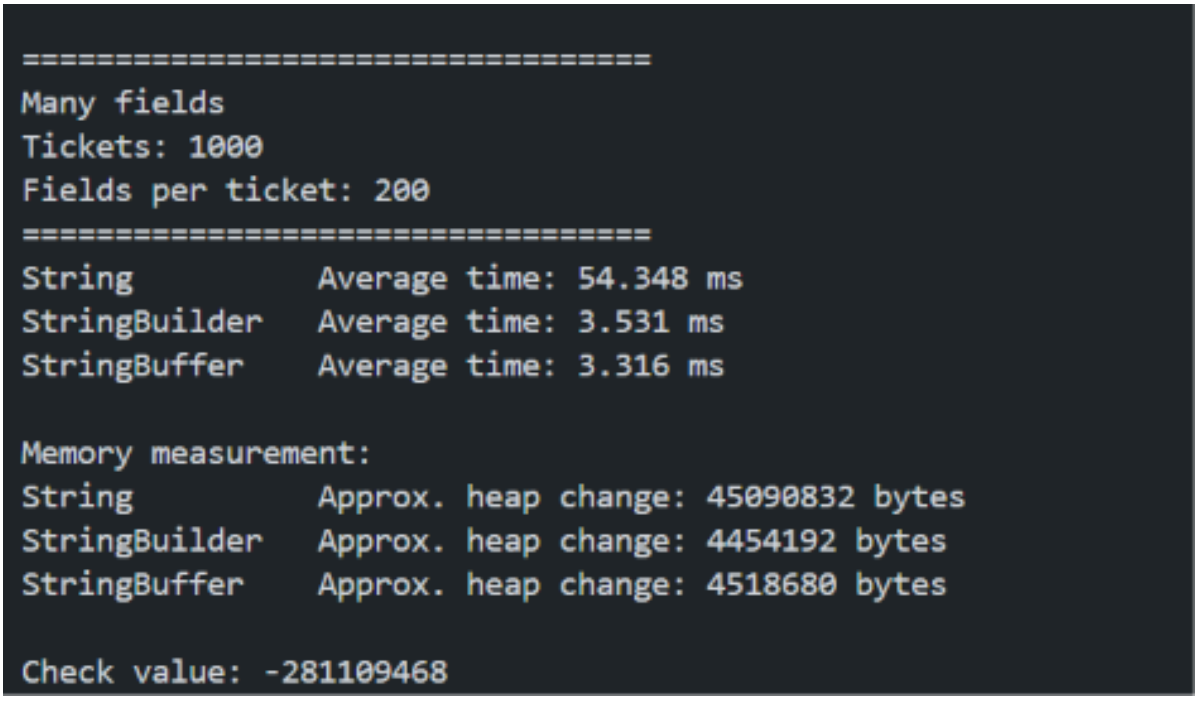


Figure 2.1. Q1 first experimental run output.

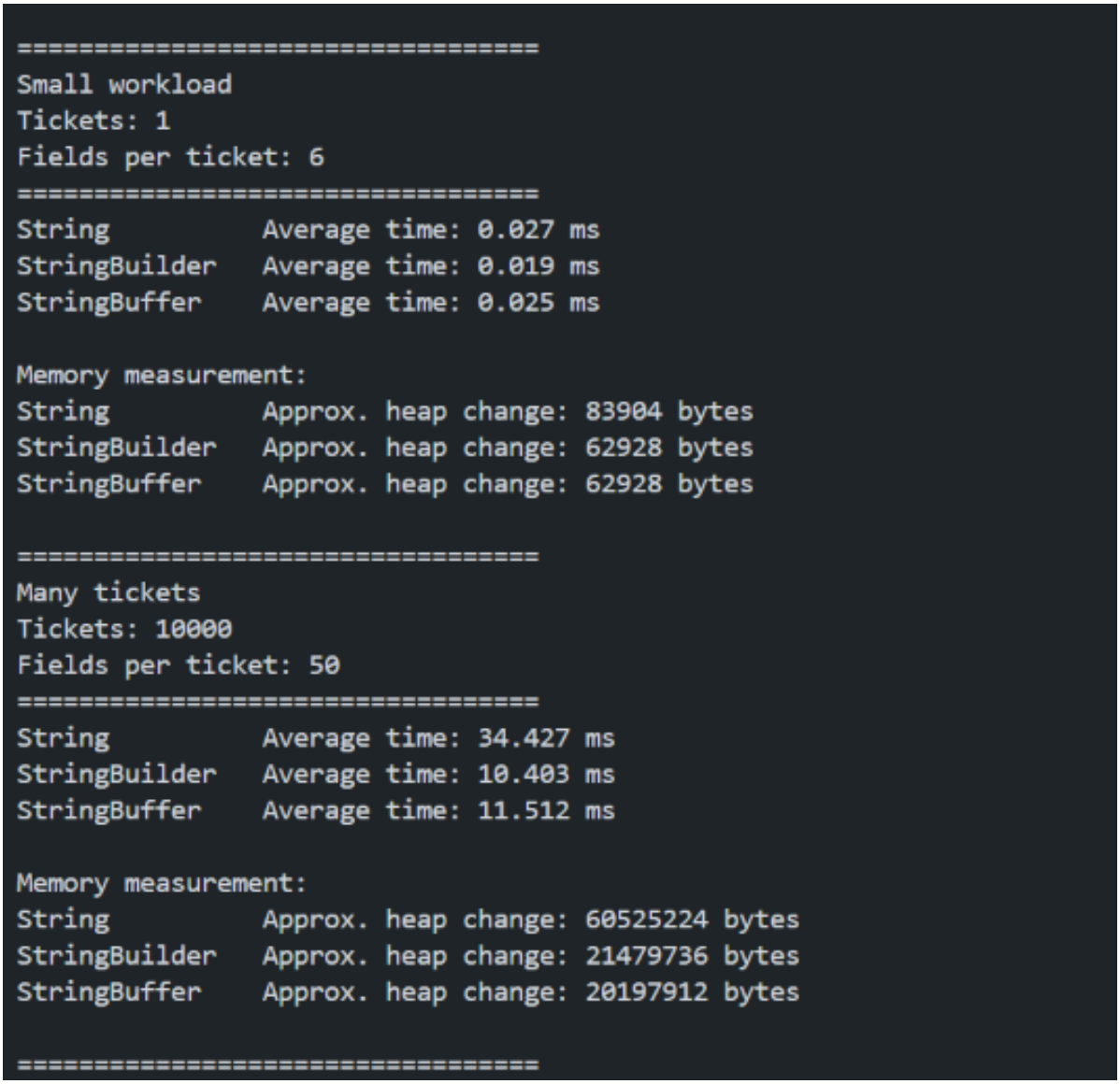


Figure 2.2. Q1 second experimental run output.

3. Q2 – When Is ArrayList an Appropriate Engineering Choice? (4 Marks)

Scenario: A music playlist supports adding songs, insertion and removal at positions, indexed access and iteration. The experiment compares ArrayList, LinkedList and ArrayDeque at three playlist sizes.

3.1 Experimental setup and observations

The initial run used playlist sizes of 1,000, 10,000 and 50,000. The displayed times below are from that run. They should be treated as one program run containing repeated measurements, not as independent benchmark trials.

Size	Collection	Add end	Insert front	Insert/remove middle	Remove front	Get middle	Iterate
1,000	ArrayList	0.017	0.190	0.057	0.079	0.552	0.436
1,000	LinkedList	0.029	0.052	0.718	0.020	54.741	0.217
1,000	ArrayDeque	0.081	0.069	N/A	0.051	N/A	1.003
10,000	ArrayList	0.033	0.310	0.291	0.285	0.204	0.417
10,000	LinkedList	0.025	0.034	7.241	0.021	956.066	0.626
10,000	ArrayDeque	0.012	0.007	N/A	0.009	N/A	1.220

Size	Collection	Add end	Insert front	Insert/remove middle	Remove front	Get middle	Iterate
50,000	ArrayList	0.026	2.048	1.907	1.882	0.120	2.304
50,000	LinkedList	0.029	0.038	49.586	0.030	3950.958	6.038
50,000	ArrayDeque	0.046	0.004	N/A	0.094	N/A	8.627

All timing values are milliseconds. ArrayDeque does not support indexed get or insertion at an arbitrary middle position, so those operations are not applicable.

3.2 Interpretation and complexity

Operation	ArrayList	LinkedList	ArrayDeque
Add at end	O(1) amortized	O(1)	O(1) amortized
Insert at front	O(n)	O(1)	O(1)
Insert at middle	O(n)	O(n) to locate position	No indexed middle operation
Remove at front	O(n)	O(1)	O(1)
Get kth element	O(1)	O(n)	No indexed access
Iteration	O(n)	O(n)	O(n)

The measured LinkedList indexed-access time increased sharply with playlist size. ArrayList provides direct indexed access. LinkedList can insert or remove efficiently at a known node position, but locating a position by index still requires traversal. ArrayDeque is designed for efficient operations at both ends.

3.3 Engineering decision and alternative

ArrayList is a reasonable choice for a normal playlist when users frequently jump to a song by index and iterate through the list. For an Up Next queue with frequent additions at both ends and removals from the front, ArrayDeque is more appropriate.

A plain array also provides O(1) indexed access, but insertion or removal requires shifting elements. A dynamically resized array is needed if capacity must grow.

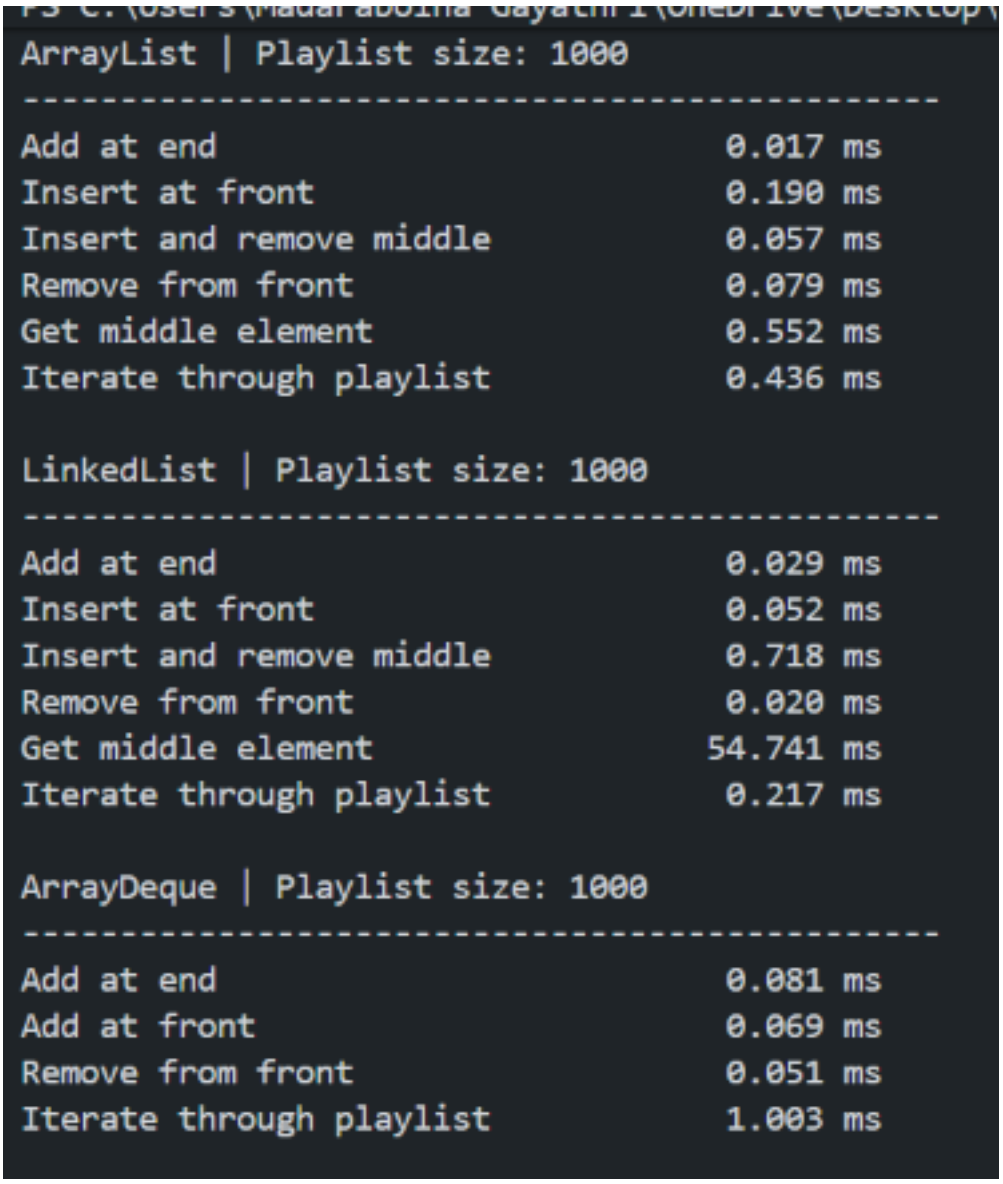


Figure 3.1. Q2 playlist-size 1,000 output.

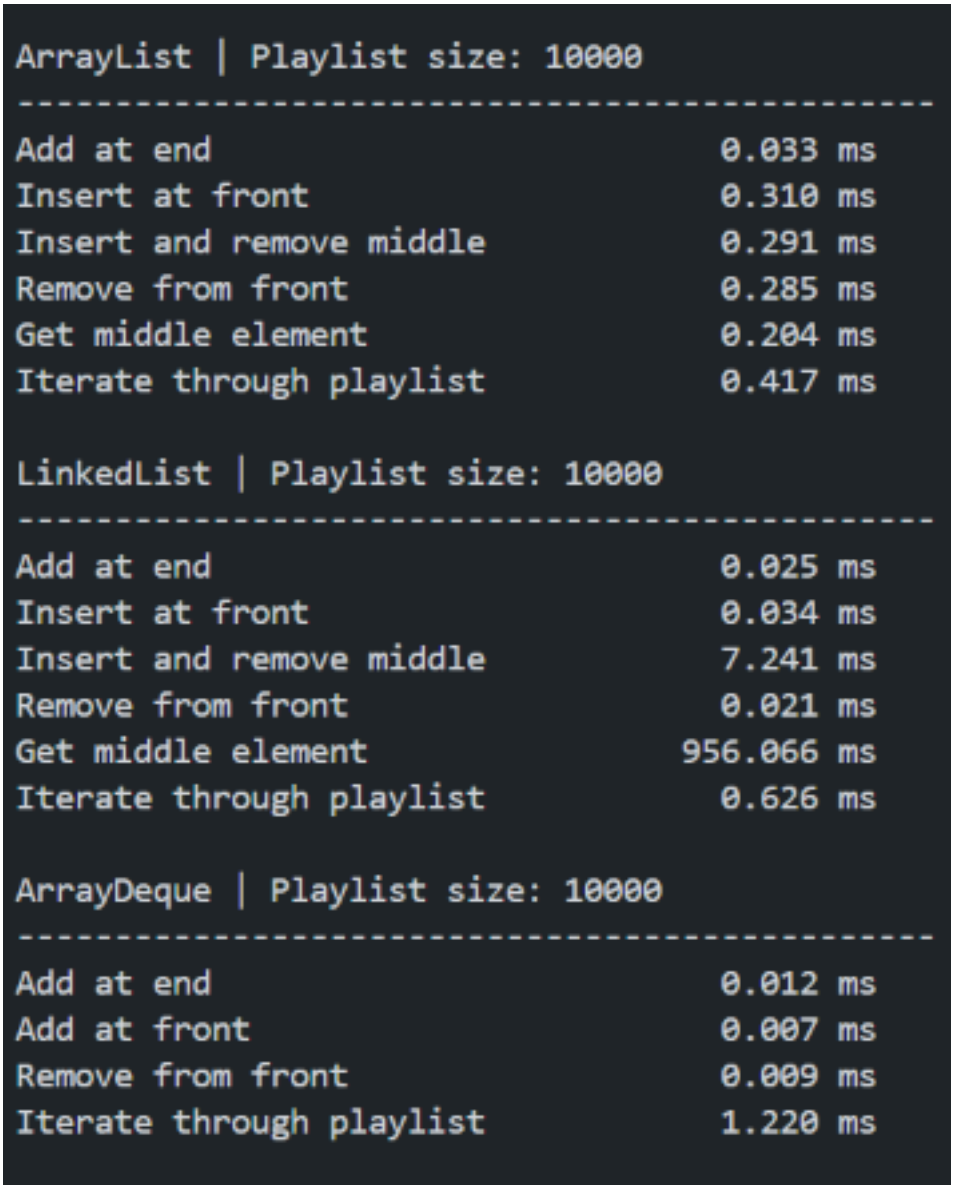


Figure 3.2. Q2 playlist-size 10,000 output.

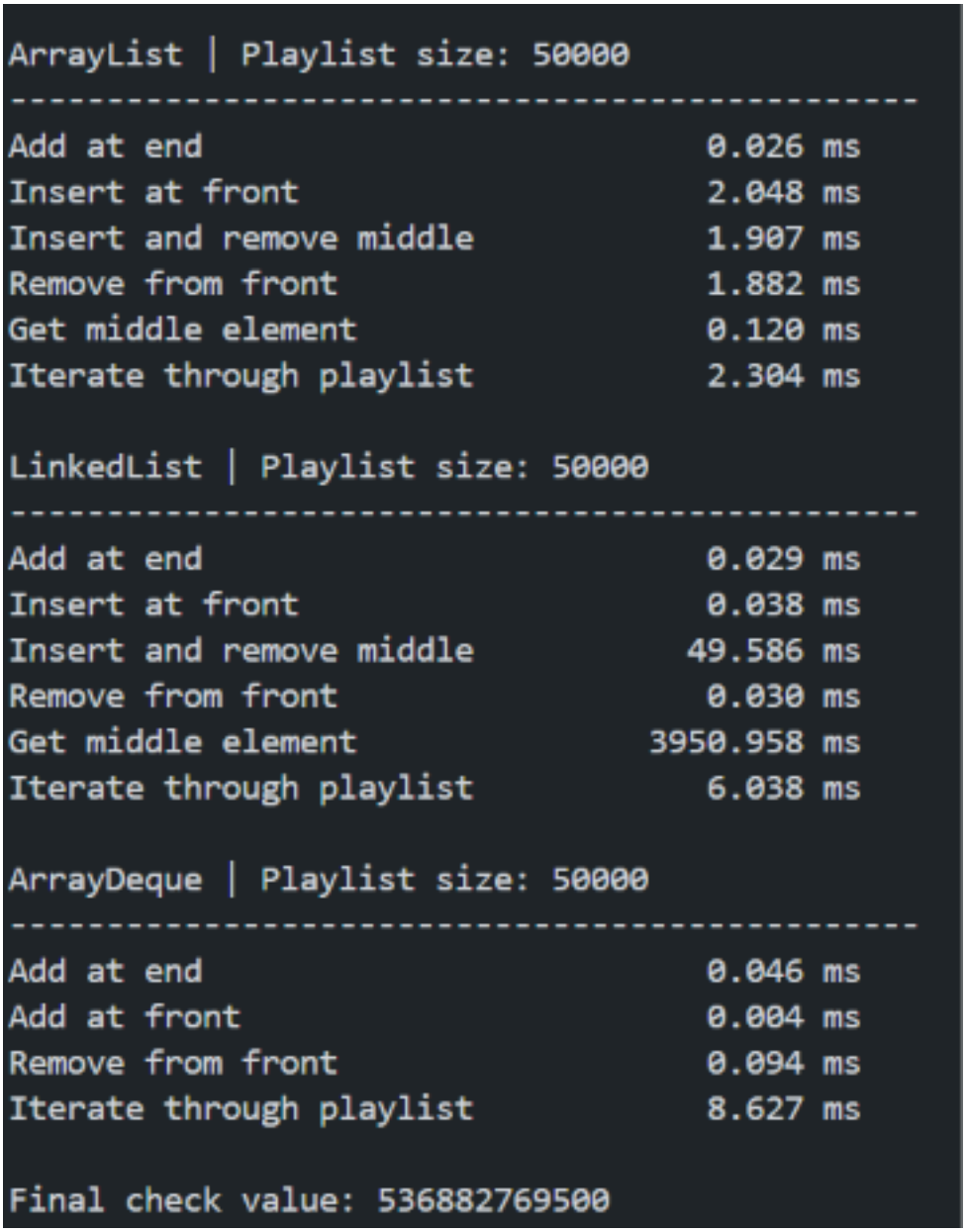


Figure 3.3. Q2 playlist-size 50,000 output.

4. Q3 – Engineer the hashCode–equals Contract (6 Marks)

Scenario: A university uses HashSet<Student> to remove duplicate attendance entries. Student equality is defined using registration number.

Test	Observed result	Interpretation
Same object added twice	Set size = 1	The same object reference is not stored twice.
Two different objects with same regNo; equals() only	Set size = 2	Without a consistent hashCode(), equal objects may be placed in different buckets.
Correct equals() and hashCode()	Unique students: 2	The duplicate registration number is correctly detected.
Change regNo after insertion	contains(): true before; false after	Mutating a field used by hashCode() can make lookup fail.

4.1 Contract explanation

If two objects are equal according to equals(), they must return the same hash code. HashSet first uses the hash code to identify a bucket and then uses equals() to distinguish objects within that bucket. A hash collision is allowed; equal objects must not produce inconsistent hash codes.

4.2 Correct implementation

The Student class overrides equals() using regNo and implements hashCode() using the same regNo field. Fields used in equality and hashing should remain stable while the object is stored in a hash-based collection.

4.3 Alternative and complexity

Structure	Average/typical operation	Benefit
HashSet<Student>	O(1) average add/contains	Fast de-duplication; requires correct equals/hashCode
TreeSet<Student> with Comparator	O(log n) add/contains	Maintains sorted order; comparator must be consistent
HashMap<String,Student>	O(1) average put/get	Registration number is the direct key; duplicate keys replace or reject based on application logic

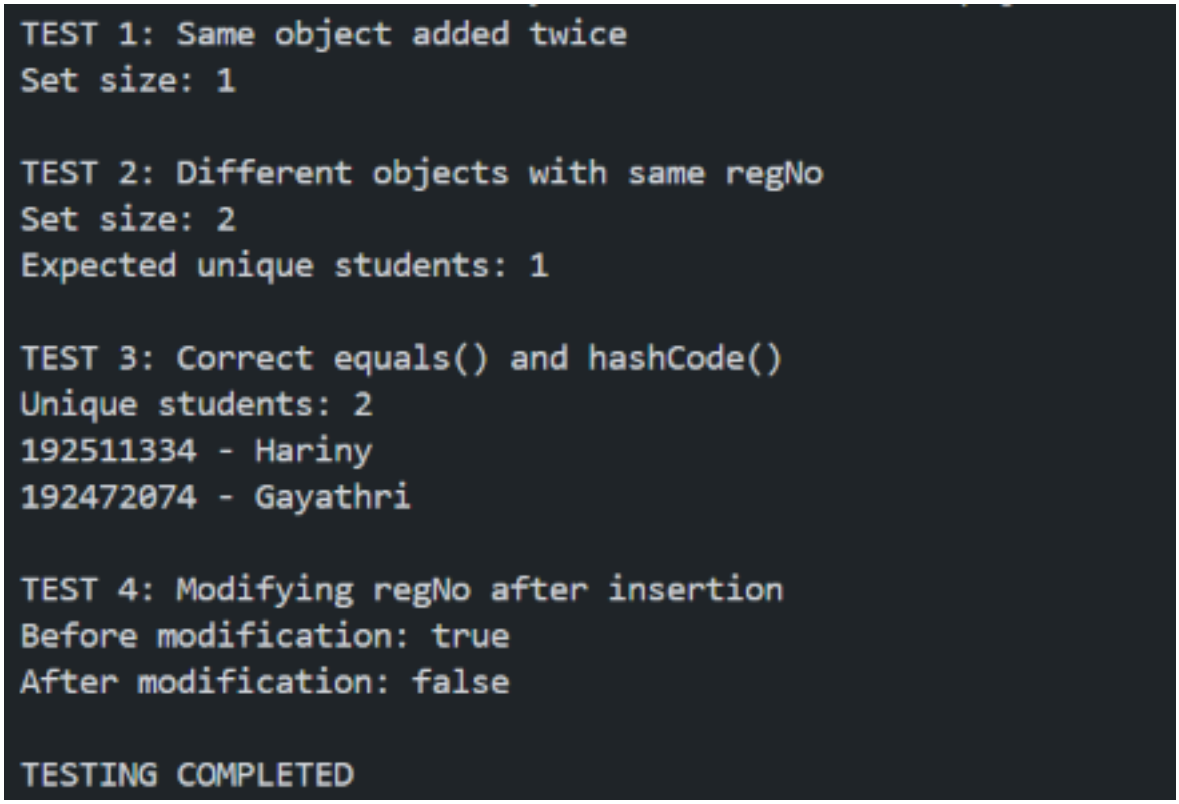


Figure 4.1. Q3 HashSet contract test output.

5. Q4 – Choosing a Map for a Pharmacy Inventory (6 Marks)

Scenario: A pharmacy stores medicine stock by medicine code and needs fast lookups, a code-sorted report and an insertion-order report. The experiment compares HashMap, TreeMap and LinkedHashMap.

Inventory size	Map	100,000 lookup batch (ms)	Report generation (ms)	Sorted by code?
1,000	HashMap	2.754	2.172	False
1,000	TreeMap	12.386	1.614	True
1,000	LinkedHashMap	5.157	0.245	True

Inventory size	Map	100,000 lookup batch (ms)	Report generation (ms)	Sorted by code?
10,000	HashMap	2.097	2.128	False
10,000	TreeMap	17.039	2.444	True
10,000	LinkedHashMap	4.376	2.234	True
50,000	HashMap	5.004	7.045	False
50,000	TreeMap	35.891	7.448	True
50,000	LinkedHashMap	11.722	8.328	True

The LinkedHashMap order test reports true because the test inserted sequential medicine codes. LinkedHashMap preserves insertion order; it does not generally sort keys. TreeMap maintains key order. HashMap does not guarantee iteration order.

5.1 Complexity and design decision

Map	put/get	Iteration/report	Ordering
HashMap	O(1) average	O(n)	No guaranteed order
TreeMap	O(log n)	O(n)	Sorted by key
LinkedHashMap	O(1) average	O(n)	Insertion order

The experiment used a target of less than 100 ms for the batch of 100,000 lookups as the chosen acceptability criterion. All observed batch times were below that target. The fastest observed lookup batches were HashMap, while TreeMap naturally provides the code-sorted report. A deployment can use separate structures or sort a snapshot when both lookup speed and sorted reporting are required.

Hashtable is a legacy synchronized map. For ordinary non-concurrent use, HashMap is generally preferred; for concurrent access, ConcurrentHashMap is a modern alternative.

5.2 Alternative sorted array

A sorted array can use Arrays.binarySearch() for O(log n) lookup. Inserting an item may require shifting O(n) elements. Producing a code-order report is O(n) because the array is already sorted. It can be memory-efficient for mostly static inventories but is less convenient for frequent updates.

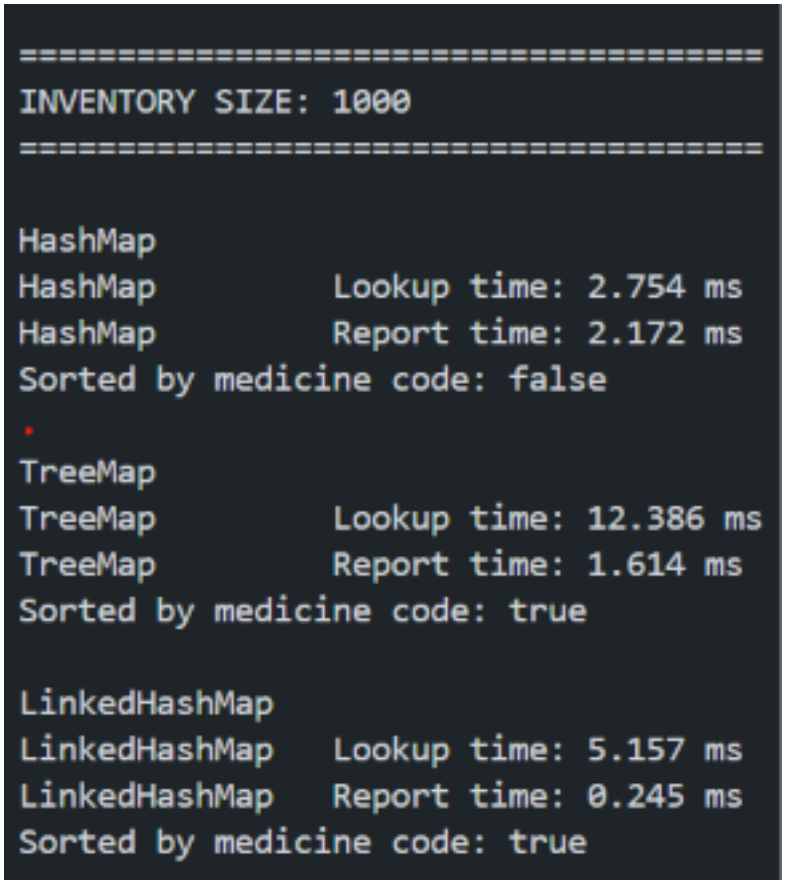


Figure 5.1. Q4 inventory size 1,000 output.

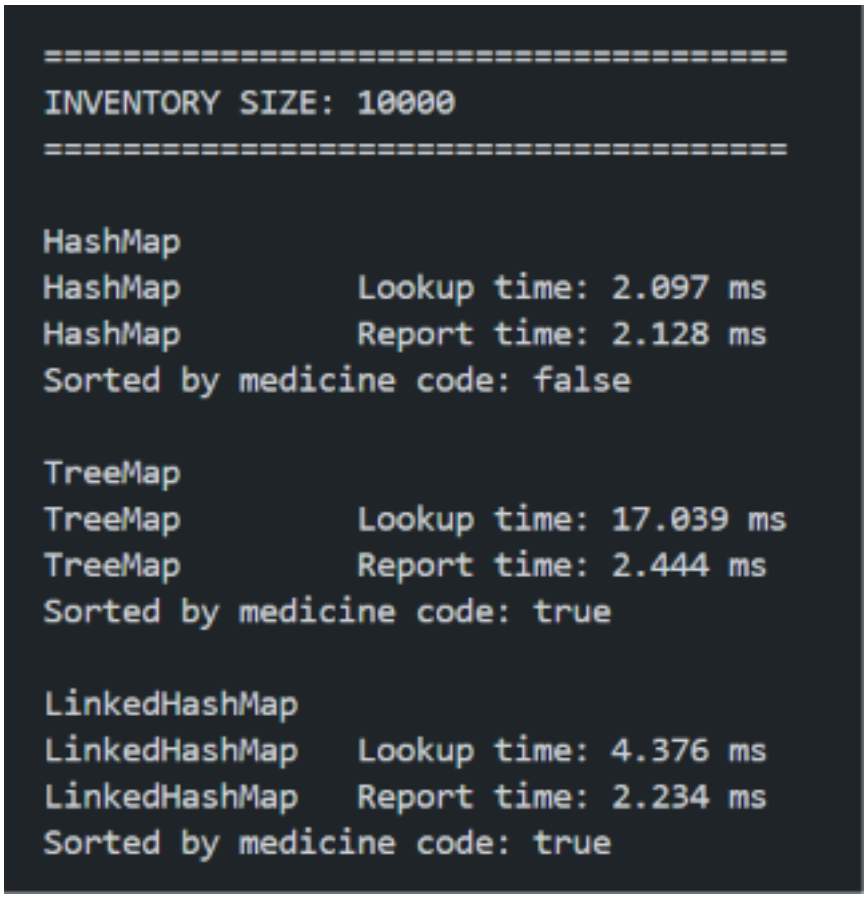


Figure 5.2. Q4 inventory size 10,000 output.

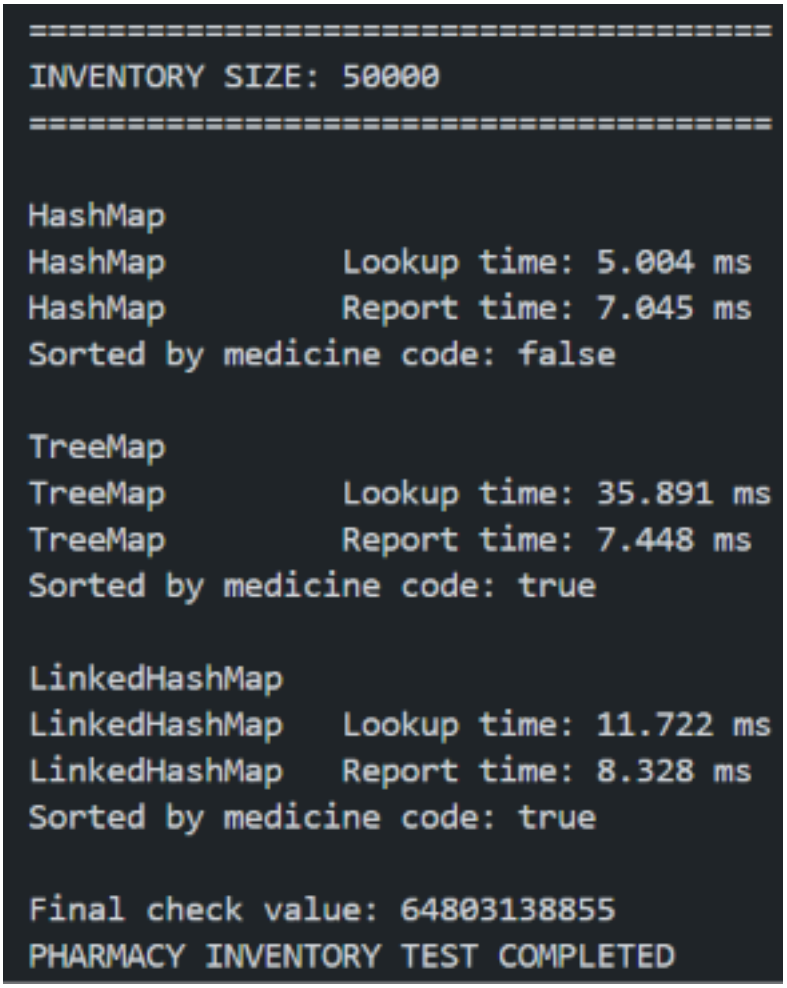


Figure 5.3. Q4 inventory size 50,000 output.

6. Q5 – Derive a Generic Ranking Component from a Principle (7 Marks)

Scenario: An inter-college sports leaderboard ranks teams by points descending, goal difference descending and team name ascending.

The implementation uses a reusable Leaderboard<T> class with a Comparator<? super T>. The method accepts List<? extends T>, copies the input list and sorts the copy, leaving the original list unchanged.

Observed ranked output: [Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1)]. The tie-break test reports Bisons before Panthers. Antisymmetry and transitivity checks both returned true.

6.1 Comparable versus Comparator

Comparable	Comparator
Defines natural ordering in the class	Defines an external/custom ordering
Uses compareTo()	Uses compare()
Usually one natural ordering	Supports multiple ranking policies
Can couple Team to one ranking rule	Allows reusable ranking strategies

Comparator is appropriate because the leaderboard may later rank players, colleges and quiz teams using different rules. The sorting operation takes O(n log n) time and the copied list requires O(n) additional space.

An alternative is Comparable<Team> if the team ranking is the single natural ordering. Another alternative is stable multi-pass sorting: sort by name, then goal difference descending, then points descending, relying on stability. This performs multiple sorting passes and is less direct than comparator chaining.

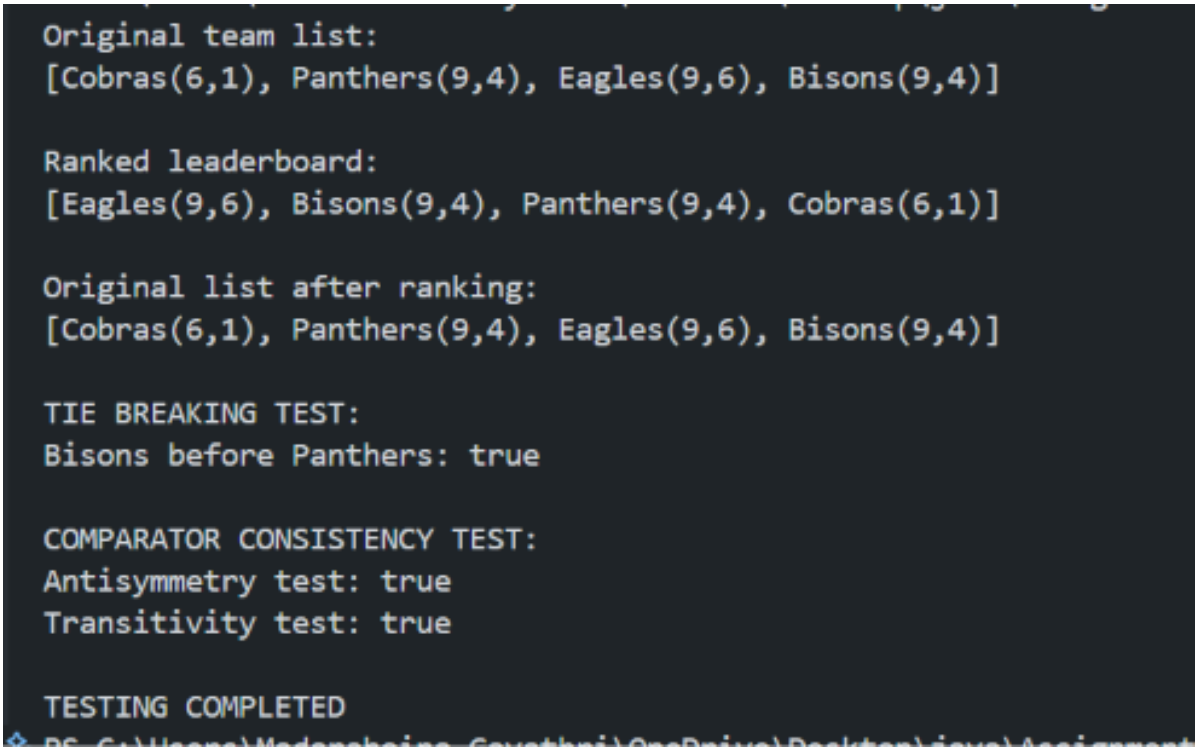


Figure 6.1. Q5 generic leaderboard and comparator validation output.

7. Q6 – Priority, Order and Fairness in a Queue (8 Marks)

Scenario: A hospital triage system treats severity S1 before S2, and so on through S5. Patients with equal severity must be treated in arrival order.

The PriorityQueue comparator orders first by severity ascending and then by an arrival counter ascending. This implements FIFO fairness within each severity level.

Treatment order	Patient
1	Meena (S1)
2	Kumar (S1)
3	Lata (S2)
4	Ravi (S3)
5	John (S3)
6	Asha (S5)

The separate equal-priority test returned Patient-A, Patient-B and Patient-C in arrival order. Patients are retrieved in priority order by repeated poll(); iterating over a PriorityQueue does not guarantee sorted order.

7.1 Complexity and starvation

Approach	Add	Remove next	Notes
PriorityQueue	O(log n)	O(log n)	Peek is O(1); priority ordering
Sorted LinkedList	O(n)	O(1)	Insertion finds the correct position
Five FIFO queues	O(1)	O(1) per queue operation	Select the highest non-empty severity queue

Starvation can occur if lower-priority patients wait indefinitely while higher-priority patients continually arrive. An aging policy can increase waiting patients' effective priority over time, but clinical safety rules must ensure critical cases are never delayed.

For a hospital handling approximately 5,000 patients per day, PriorityQueue with an arrival counter is a reasonable starting design, subject to clinical validation, concurrency control and audit requirements.

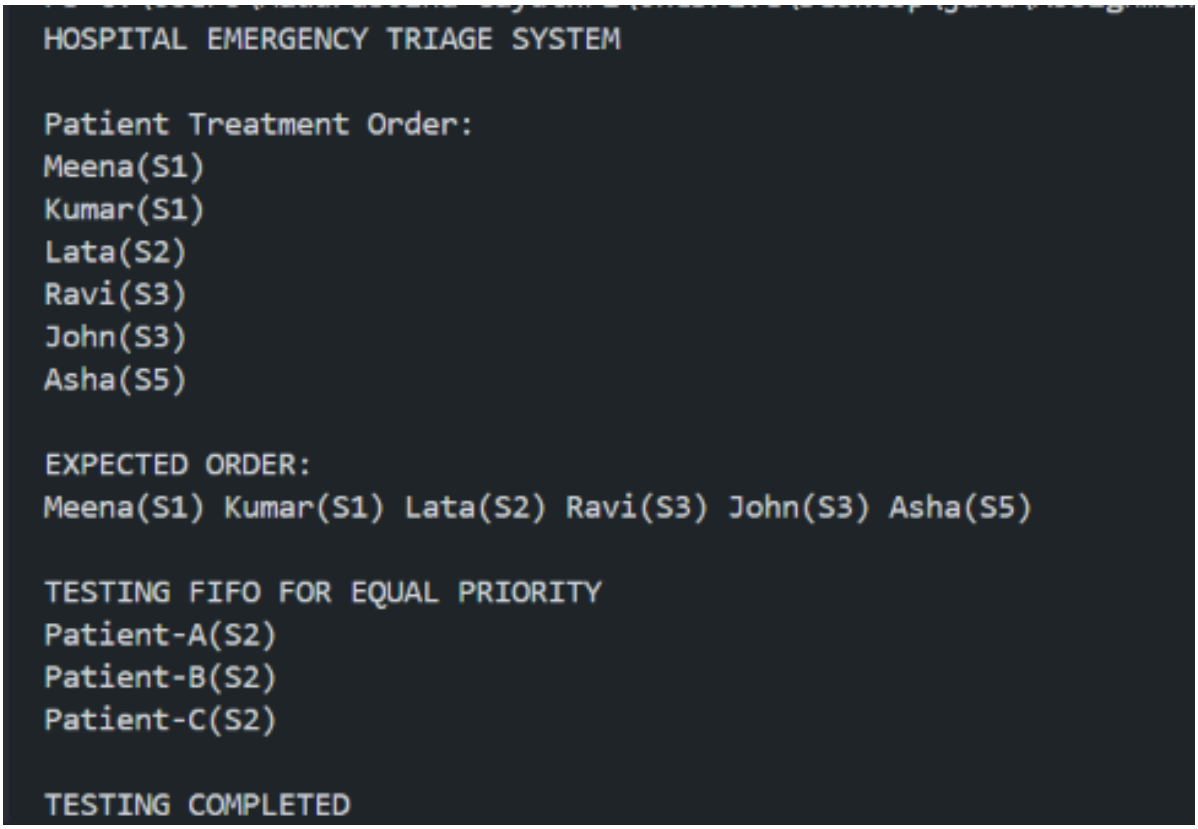


Figure 7.1. Q6 hospital triage treatment order and FIFO test.

8. Q7 – Day-4 Design Challenge: Diagnose, Redesign and Validate (13 Marks)

The design challenge uses the Q2 playlist evidence. The original design uses LinkedList for a workload that performs frequent indexed song retrieval. The diagnosed limitation is O(n) traversal for get(k). The redesign uses ArrayList, which provides O(1) indexed access.

8.1 Original evidence and diagnosis

Playlist size	LinkedList lookup average (ms)	ArrayList lookup average (ms)	Reported speedup
1,000	0.441	0.054	8.22×
10,000	7.554	0.048	157.77×
50,000	18.140	0.048	376.51×

The redesign benchmark performed 1,000 lookups per measured run, used two warm-up runs and five measured repetitions, and used the same generated search positions for both implementations. The output confirmed that both playlists contained the same songs.

8.2 Design objective and revised solution

Objective: reduce the time for repeated indexed song retrieval while preserving playlist contents and order. Replace LinkedList with ArrayList for the indexed-access workload. ArrayList stores elements in an array and

calculates the location of an indexed element directly, avoiding node traversal.

Pseudocode: create identical LinkedList and ArrayList playlists; generate a shared set of random indices; warm up both; measure repeated get(index) operations; calculate average time and speedup; verify both lists contain identical songs.

8.3 Prediction, validation and trade-offs

Prediction: ArrayList should improve indexed retrieval, especially as playlist size increases, because its indexed access is O(1) while LinkedList indexed access is O(n). The measured results support this prediction. The reported speedup rose from 8.22× at 1,000 songs to 376.51× at 50,000 songs in the supplied run.

Trade-off: ArrayList insertion and removal near the front or middle require shifting elements and can take O(n). LinkedList may still be appropriate when frequent insertions/removals occur at known node positions and indexed retrieval is uncommon. Therefore, the redesign is justified for a playlist dominated by random indexed access, not as a universal replacement.

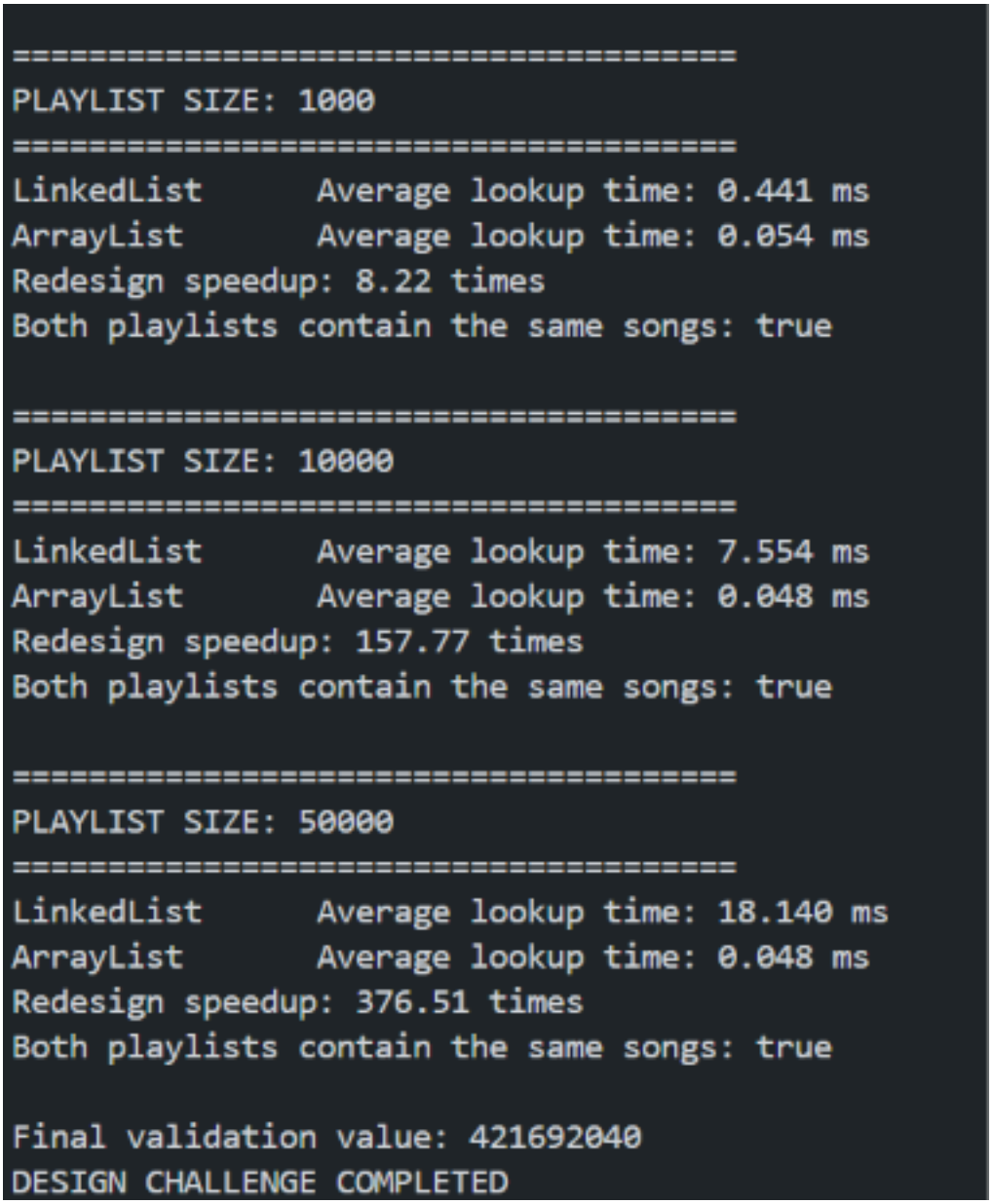


Figure 8.1. Q7 original-versus-redesigned lookup benchmark output.

9. Overall Conclusion

The experiments demonstrate that data-structure selection should follow the expected operation pattern rather than a general claim that one collection is always faster. StringBuilder is suitable for repeated single-threaded text construction; ArrayList is suitable for indexed playlist access; HashSet requires a correct equals/hashCode contract; TreeMap provides sorted map keys; Comparator supports reusable ranking policies; PriorityQueue supports severity-based scheduling with explicit FIFO tie-breaking; and ArrayList provides a validated redesign for the indexed-access playlist workload.

All timing values are observations from the supplied environment and should not be treated as universal performance guarantees. For a final submission, retain the source code, raw output, screenshots and progressive Git commits, and be prepared to explain the assumptions and limitations.

10. References

- Oracle. Java Platform, Standard Edition API Specification – String, StringBuilder and StringBuffer.
- Oracle. Java Platform, Standard Edition API Specification – Collections Framework, List, Set, Map, Queue, Deque and PriorityQueue.
- Oracle. Java Platform, Standard Edition API Specification – Comparator, Comparable, HashMap, TreeMap and LinkedHashMap.
- Course handout: Programming in Java, Assignment 2 – CO2 (provided assignment document).